

Usage Guide

Starting

To annotate from scratch, use the following command:

```
./annotation_agreement.py 3sample-empty.json
```

Controls

h	j	k	l	move inside single annotation section
	m	M		move between annotation sections
	n	N		move between examples
	enter			mark selected annotation as correct
	backspace			mark all annotations as incorrect
	=			reset annotations for selected category
ctrl	+	s		save (but it also auto-saves when moving between items)

Annotation categories

Section 1, “categories”

core_reason_for_answer should be one of the following options:

- **code**, if the agent answered mainly based on code output.
- **raw data**, if the agent answered mainly based on “eyeballing” the raw data.
- **question format**, if the agent answered mainly based on strategizing about the exam format.
- **other**, if none of the options above works.

core_problem_solving_strategy should be one of the following options, based on the primary strategy of the agent:

- **statistical test**, if the primary strategy was to perform a formal hypothesis test or compute a test statistic and p-value (examples: Augmented Dickey–Fuller, KPSS, Granger causality, t-test, Wilcoxon test, chi-square test, permutation test).
- **spectral analysis**, if the primary strategy was to analyze frequency content of the series (examples: FFT, periodogram, Welch’s method, Lomb–Scargle, bandpower, spectral peak detection).
- **curve fitting**, if the primary strategy was to fit a parametric model to data (examples: least-squares polynomial/regression fitting, AR/ARMA/ARIMA/SARIMA parameter estimation, nonlinear curve fits, sklearn or statsmodels model.fit usage, model-based forecasting).
- **windowed/rolling statistics**, if the primary strategy was to compute statistics over sliding or rolling windows, without transforming into the frequency domain (examples: rolling mean/variance, rolling correlations or cross-correlations, rolling regression, windowed autocorrelation, rolling volatility).
- **simple arithmetic**, if the primary strategy consisted of direct, non-model-based summary calculations without statistical inference or signal analysis (examples: global mean/median/variance, counts, ratios, totals, expanding statistics, basic differencing).
- “other”, if none of the above fits.

Section 2, “methodological problems”

conceptual_misunderstanding should be true if the agent demonstrates a fundamental misunderstanding of key concepts relevant to the question.

wrong_core_problem_solving_strategy should be true if the strategy chosen by the agent (“core_problem_solving_strategy”) was not suitable for answering the question. For example, if the model decided to use curve fitting, when using a statistical test would have been better.

wrong_method_within_strategy should be true if the strategy chosen by the agent (“core_problem_solving_strategy”) was suitable for answering the question, but failed due to the specific method chosen within the strategy. For example, the model selected an incorrect statistical test, heuristic, threshold, or algorithmic variant.

unsupported_assumption should be true if for the method that the agent chose, it made an unjustified assumption about the data or the options. For example, assuming stationarity without evidence.

implementation_errors should be true if there were coding or algorithmic mistakes that affect the validity of the result. Code execution errors are that caused the code to fail are not included in this category.

incorrect_result_interpretation should be true if the agent incorrectly interprets the result of any of his computations. For example, the model wrongly conclude that the difference is statistically significant.

insufficient_evidence_guess should be true if the agent answered whilst not having gathered sufficiently convincing evidence.

Section 3, “code_problems”

code_result should be one of the following options, indicating the overall success of the code the model produced (if any):

- **success**, if the agent managed to obtain the data it intended. If the code initially failed due to errors, but the agent managed to fix the errors, it still counts as a success.
- **partial failure**, if the agent failed to obtain some of the desired data, but managed to get at enough results to produce an answer. For example if the code failed after printing sufficient evidence.
- **complete failure**, if the agent failed to obtain the results and gave up or ran out of turns.
- **no code**, if the agent didn’t produce any code.

tool_usage_trouble should be true if the agent had some trouble following instructions to use the tool correctly. This mainly includes cases where the agent generated an improper coding tool call or had trouble calling the tool properly. Regular errors and exceptions that caused the code to fail are not included in this category.

other should be true if there was some other issue with the code that does not fall into any of the other categories.

Section 4, “other_problems”

reasoning_answer_mismatch should be true if the agent settled on one answer in the thinking trace and then answered differently.

reasoning_tool_usage_mismatch should be true if the agent claims it will use the coding tool (such as saying “let’s compute”, or crafting a code solution in the thinking trace) but then goes straight to answering.

hallucinated_values_in_reasoning should be true if the agent introduces numerical values that do not appear in the data or the code output. Reasonable rounding does not count.

Creating the original data files

To create the initial save files, use (in nushell):

```
./annotation_agreement.py ... (ls llm-judge-outputs--3sample | get name) -o 3sample
```